

05 • Dot Products, Softmax & Attention Math

Source: [fanout.sh / inference-eng / 00-03](https://fanout.sh/inference-eng/00-03) • ~11 min

IN SHORT

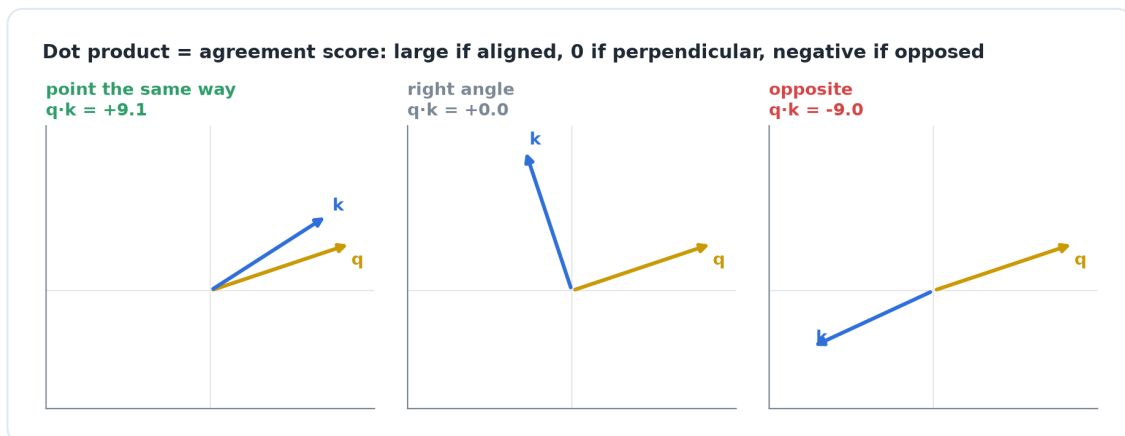
To write one new token at 4,096 tokens of context, Llama 3 8B's attention does about **4.2 million dot products** (~2.1B operations) but must read about **0.5 GB** of stored keys and values. That's ~4 operations per byte, far below the ~300 an H100 needs, so **decode attention is memory-bound**. The K/V data grows with every token and every user until it outweighs the model. Softmax needs the whole row, which is why fast kernels use a running (online) softmax.

1. The dot product: an agreement score

Multiply two equal-length lists pair by pair, then add:

$$(2, -1, 0.5) \cdot (3, 4, 2) = 6 - 4 + 1 = 3$$

- In attention each list has **128 numbers**: 128 multiplies + ~128 adds $\approx$ **256 operations**.
- The result says how much two vectors **point the same way**:



Aligned vectors give a large positive score, perpendicular ones zero, opposed ones negative.

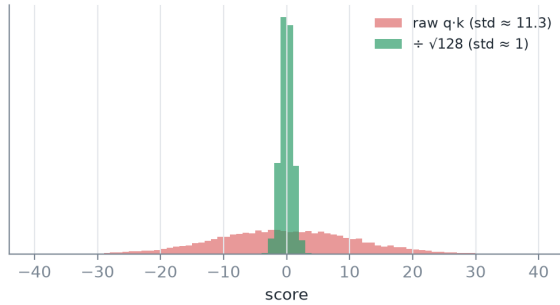
- Attention uses it as a **relevance score**: the current token's **query** asks, each past token's **key** answers, and $q \cdot k$ says how well they match.

Why divide by $\sqrt{128}$

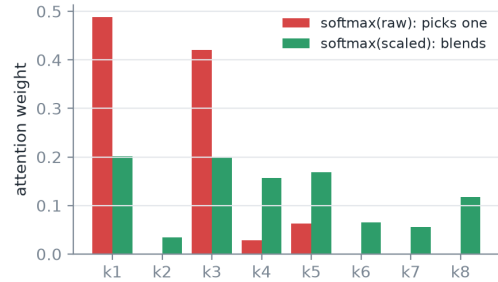
If query and key entries are random with mean 0 and variance 1, their dot product has **variance = length** (footnote in *Attention Is All You Need*). For 128 numbers, scores spread to about **± 11** , and softmax then puts nearly all weight on one token: attention stops blending and just picks.

Why divide by $\sqrt{128} \approx 11.3$: keep scores near ± 1 so attention can mix

Random 128-d q, k: variance = 128



Same 8 keys, two softmaxes



Simulated: raw scores spread ± 11 and softmax picks a winner; dividing by $\sqrt{128} \approx 11.3$ brings them to ± 1 and the weights blend.

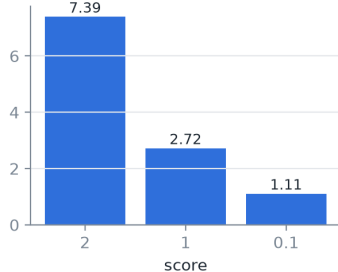
INTUITION

For inference the scaling is nearly free: one multiply per score, and fast kernels fold it into work they already do.

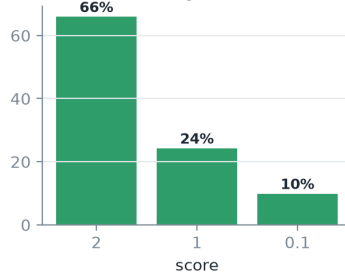
2. Softmax: scores into weights

Softmax makes scores **positive and summing to 1**: raise e to each score, divide by the total.

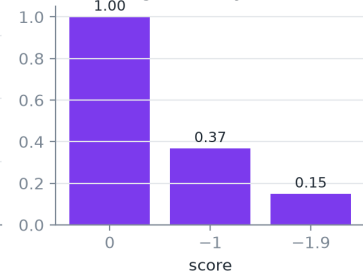
1 · e^{score} (sum ≈ 11.2)



2 · $\div$ sum = weights



stable: subtract max first
 same weights, every $e^x \leq 1$



Why subtract the max: FP16 tops out at 65,504, but $e^{12} \approx 162,755$ already overflows the whole row.

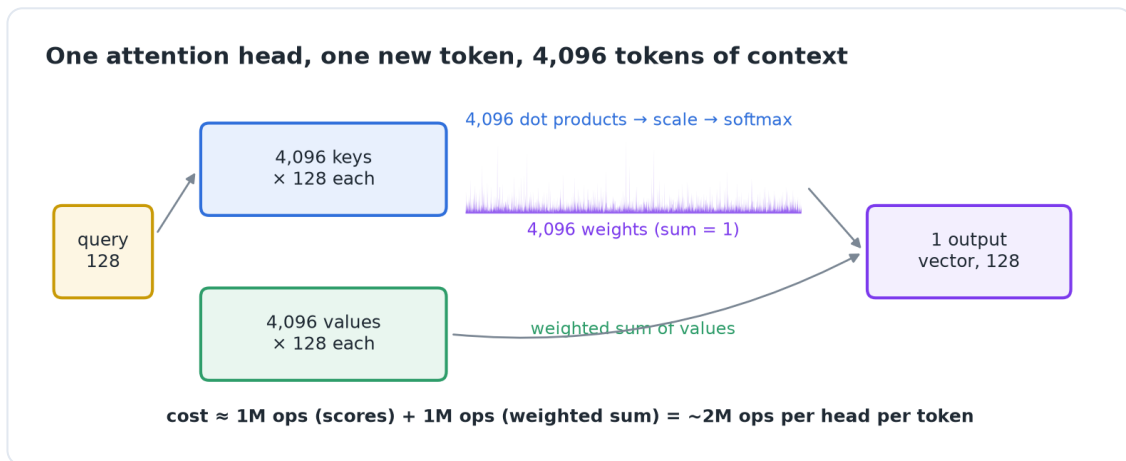
Scores 2, 1, 0.1 $\rightarrow e^x \approx 7.39, 2.72, 1.11$ (sum ≈ 11.2) $\rightarrow 66\%, 24\%, 10\%$. The biggest score wins most, but nothing gets zero.

- **Overflow trap:** FP16's largest value is **65,504**, but $e^{12} \approx$ **162,755**. One large score turns the whole row into garbage.
- **Fix:** subtract the row's **maximum** first (2, 1, 0.1 $\rightarrow$ 0, -1, -1.9). Same weights, and every exponent is ≤ 1 .

KEY IDEA

Softmax needs the **max and the sum over the entire row** before any weight is final. This shapes how every fast attention kernel is built (section 6).

3. One head, one new token



Query · 4,096 keys → scale → softmax → weighted sum of 4,096 values → one 128-number summary of the conversation, as seen from the new token.

Work: ~**1M ops** for the scores + ~**1M ops** for the weighted sum = ~**2M ops per head**.

4. The bill for one token: ops vs bytes

Scale to the whole model: **32 layers × 32 query heads × 4,096 = 4,194,304 dot products ≈ 2.1B operations**. The weight matrices cost ~**15B** ops per token, so at this context attention's arithmetic is only ~**1/8**.

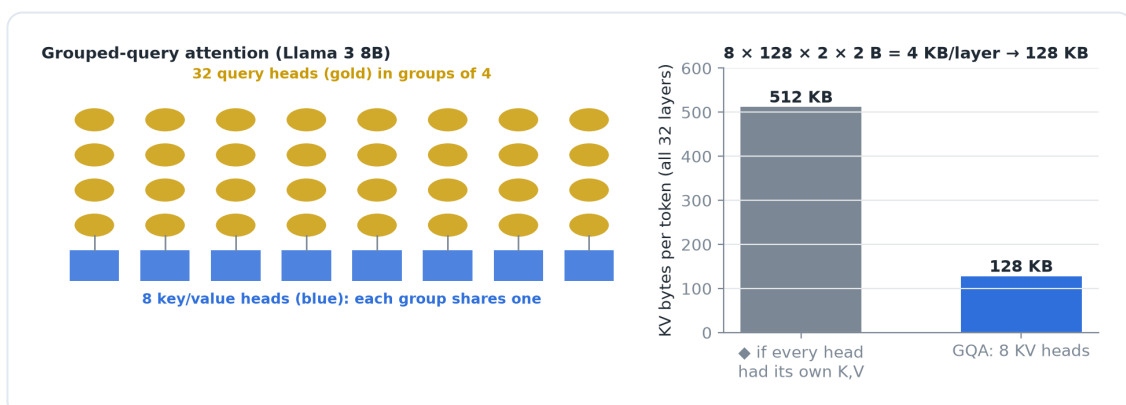
Where 2.1B comes from: two equal halves, the scores and the weighted sum of values (scaling and softmax add only a few million).

scores : 4.19M dots × 256 ≈ 1.07B values : 32 × 32 × 4096 × 128 × 2 ≈ 1.07B ⇒ ≈ 2.1B

Where ~15B comes from: each weight does one multiply and one add per token, so ~2 ops per parameter. The ~0.5B-parameter input embedding table (128,256 vocab × 4,096) is a lookup, not a matmul, so ~7.5B parameters do work:

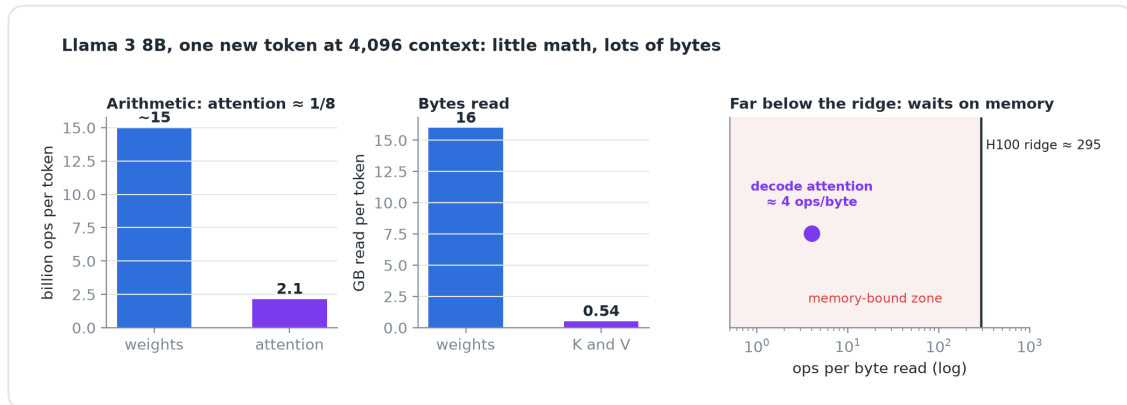
$$2 \times (8.03\text{B} - 0.53\text{B}) \approx 15\text{B} \quad \frac{2.1}{15 + 2.1} \approx 12\% \approx \frac{1}{8}$$

Now count bytes. Llama 3 uses **grouped-query attention (GQA)**: the 32 query heads share just **8** sets of keys and values.



Four query heads share each K/V head, so each layer stores 8 keys + 8 values per token.

$$8 \text{ heads} \times 128 \times 2 \text{ (K, V)} \times 2 \text{ B} = 4 \text{ KB/token/layer} \times 32 = 128 \text{ KB/token} \times 4096 \approx 0.5 \text{ GB}$$



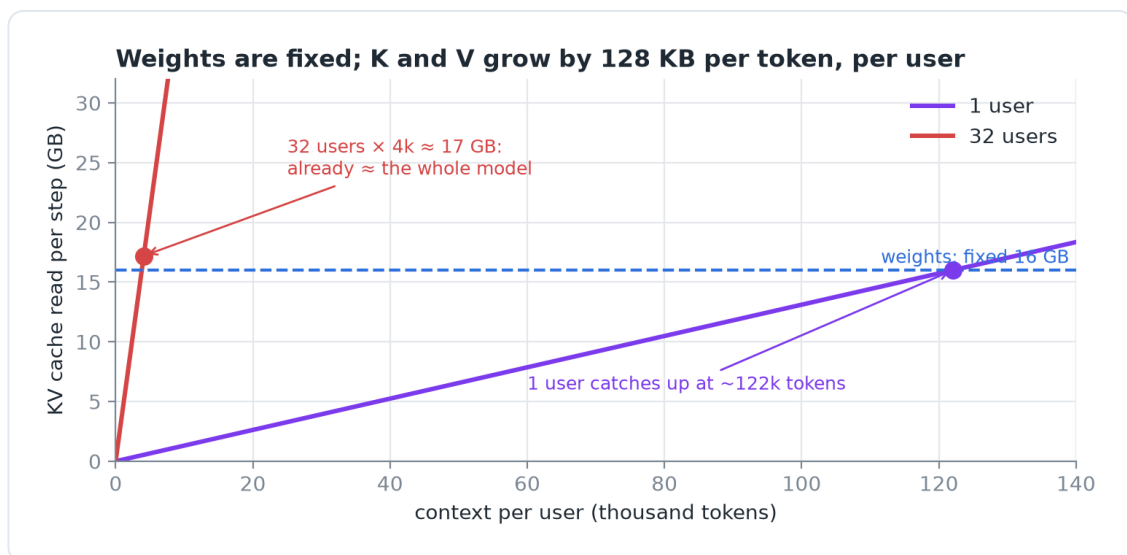
$2.1\text{B ops over } \sim 0.54\text{B bytes} \approx 4 \text{ ops/byte}$; an H100 needs ~ 295 ($989 \text{ TFLOPS} \div 3.35 \text{ TB/s}$) to keep its math units busy.

KEY IDEA

Four is nowhere near 300: **decode attention spends almost all its time waiting on memory.**

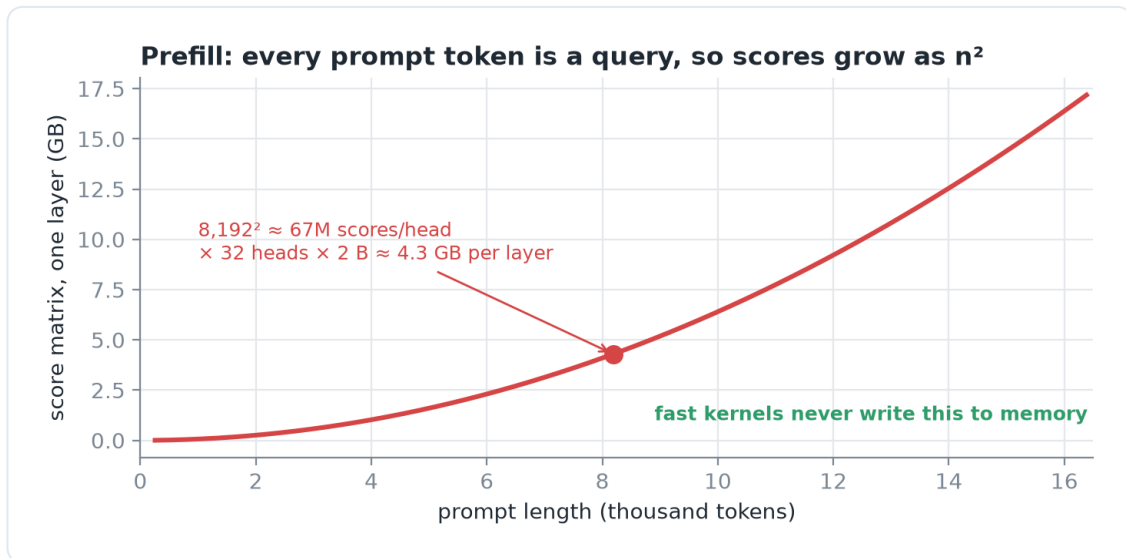
5. The memory bill grows

- **Weights:** a fixed **16 GB**, read once per step however long the conversation.
- **Keys and values:** **+128 KB per token**, per user.



One user's K/V catches the weights at $\sim 120\text{k}$ tokens; 32 users at 4k tokens each already hold $\sim 17 \text{ GB}$.

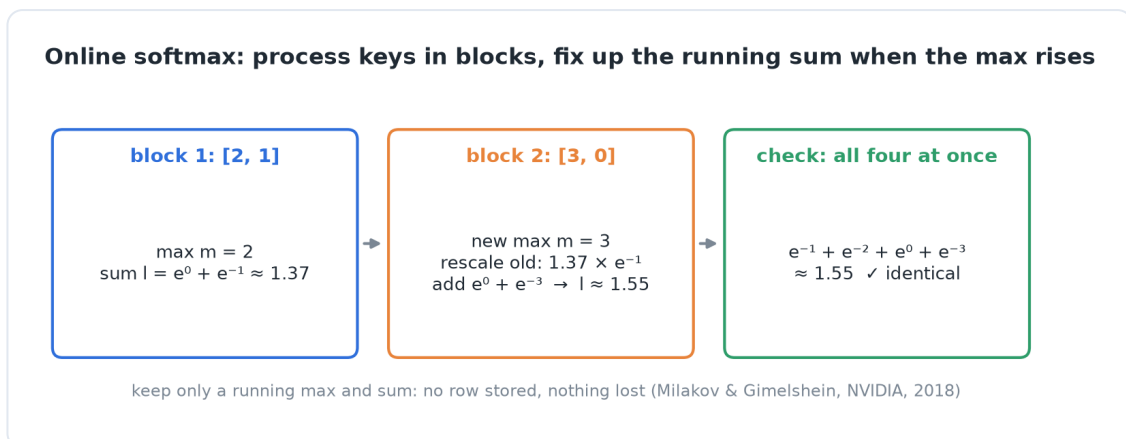
Prefill has a different problem: every prompt token is a query at once, so the scores form an **$n \times n$** square.



An 8,192-token prompt: $\sim 67\text{M scores per head}$; $\times 32 \text{ heads} \times 2 \text{ bytes} \approx 4.3 \text{ GB of scratch per layer}$.

6. Online softmax: never write the square

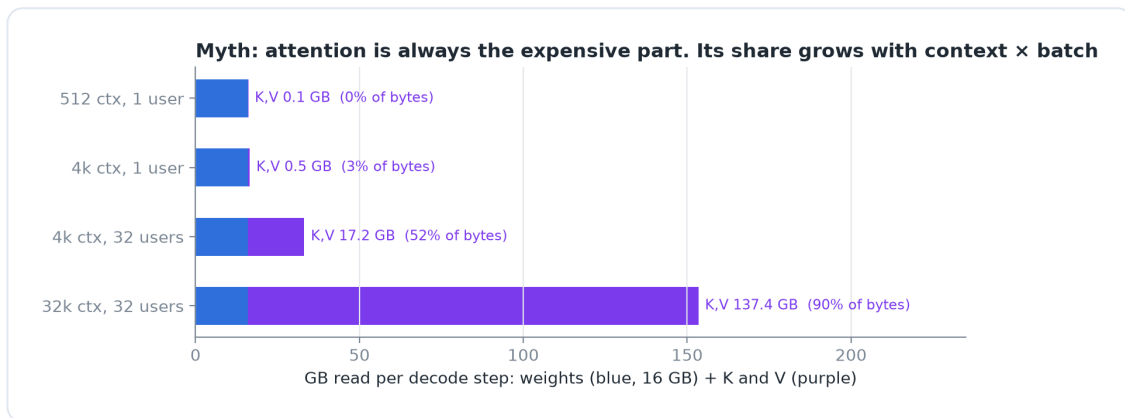
Fast attention kernels walk the keys **in blocks** and keep a **running max and running sum**. When a new block raises the max, rescale the old sum by $e^{(\text{old max} - \text{new max})}$, then add the new terms.



Blocks [2, 1] then [3, 0] give a running sum of ~ 1.55 , exactly the full-row result. Nothing stored, nothing lost.

This is the **online softmax** (two NVIDIA engineers, 2018), the foundation of **FlashAttention**; the attention module builds on it.

7. Two misconceptions



At short context and small batch the weights dominate the bytes; attention's share grows with context × batch, mostly through memory.

TRAP

- "Attention is always the expensive part": not at short contexts and small batches.
- "Softmax is cheap, so it doesn't matter": the arithmetic is cheap, but needing the whole row shapes every fast kernel.

8. Recap

Lesson 05 on one card

DOT PRODUCT

agreement score of two vectors
 ≈ 2 ops per number (256 for $d = 128$)
 divide by $\sqrt{d}$ to keep scores $\sim \pm 1$

SOFTMAX

scores $\rightarrow$ positive weights summing to 1
 subtract the max (FP16 overflows)
 needs the whole row, or a running one

DECODE ATTENTION

4.2M dots ≈ 2.1 B ops per token
 reads 0.5 GB of K,V at 4k context
 ≈ 4 ops/byte $\rightarrow$ memory-bound

GROWTH

128 KB per token per user
 1 user = weights at ~ 120 k tokens
 $32 \text{ users} \times 4k \approx 17 \text{ GB}$

Interview cheat sheet

Revise from these pages alone. ♦ = my addition beyond the video; everything else is from the lesson. Builds on 01–04.

Say it in 30 seconds

Attention scores are dot products of the new token's query with every past key, scaled by $\sqrt{d}$ and softmaxed into weights that mix the values. Per decode token that's little arithmetic but a full read of every stored key and value, about 4 ops per byte, so decode attention is memory-bound. K/V grows 128 KB per token per user in Llama 3 8B, overtaking the weights at long context or big batch. Prefill instead faces an n^2 score matrix, which FlashAttention avoids with an online softmax.

Only numbers worth memorizing

- **Llama 3 8B attention:** 32 layers, 32 query heads, 8 KV heads (GQA), head dim 128. **FP16 max:** 65,504.

Derive, don't memorize

Let L = layers, H = heads, d = head dim, n = context length.

1. Attention ops per decode token = dots $\times$ cost per dot $\times$ 2 (scores + weighted sum)

$$\text{ops} \approx L \cdot H_q \cdot n \cdot 2d \cdot 2 = 32 \cdot 32 \cdot 4096 \cdot 256 \cdot 2 \approx 2.1 \text{ B} \quad (\text{vs } \approx 15 \text{ B for weights})$$

INTUITION

The final $\times 2$ = scores + weighted sum of values. Weights: 2 ops per non-embedding param, $2 \times 7.5\text{B} \approx 15\text{B}$ (the embedding table is a lookup).

2. KV bytes per token: one K and one V per KV head per layer

$$\text{KV/token} = 2 \cdot L \cdot H_{\text{kv}} \cdot d \cdot \text{bytes} = 2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 128 \text{ KB} \quad \Rightarrow \quad \frac{2.1 \text{ B ops}}{4096 \times 128 \text{ KB}} \approx 4 \text{ ops/byte}$$

INTUITION

$4 \ll \sim 300$ ridge (01, derivation 2): memory-bound. K/V matches the 16 GB weights at $16 \text{ GB} \div 128 \text{ KB} \approx 122\text{k}$ tokens for one user, or $\sim 4\text{k}$ each for 32 users. GQA's 8 KV heads instead of 32 cut this $4\times$ ♦.

3. Prefill scores = n^2 per head

$$n^2 \cdot H \cdot 2 \text{ B} = 8192^2 \times 32 \times 2 \approx 4.3 \text{ GB per layer}$$

INTUITION

Quadratic scratch is why FlashAttention tiles keys and never writes the square to memory.

4. Online softmax: running max m and sum l , rescaled when m rises

$$m' = \max(m, m_{\text{blk}}) \quad l' = l \cdot e^{m - m'} + \sum_{\text{blk}} e^{x - m'}$$

INTUITION

Blocks [2, 1] then [3, 0]: $l = 1.37 \rightarrow 1.37 \cdot e^{-1} + e^0 + e^{-3} \approx 1.55$, identical to the full row.

Decode attention vs prefill attention

	Decode (1 new token)	Prefill (n prompt tokens)
Queries	1 per head	n per head
Problem	Reading all K/V (bytes)	$n \times n$ score matrix (scratch)
Grows with	Context $\times$ batch	Prompt length squared
Main fixes	GQA, KV quantization ♦, paged KV	FlashAttention (tiling + online softmax)

Rapid-fire Q&A

Question	Crisp answer
Why divide by $\sqrt{d}$?	Random q·k has variance d ; unscaled scores make softmax one-hot.
Why subtract the max in softmax?	FP16 overflows ($e^{12} > 65,504$); the weights are unchanged.
What does GQA save?	K/V memory and bandwidth: 8 shared KV heads instead of 32.
Is attention the bottleneck?	Only at long context or big batch; otherwise the weights dominate.

TRAP

- Counting attention FLOPs instead of K/V bytes for decode.
- Calling softmax "cheap": its whole-row dependency dictates kernel design.